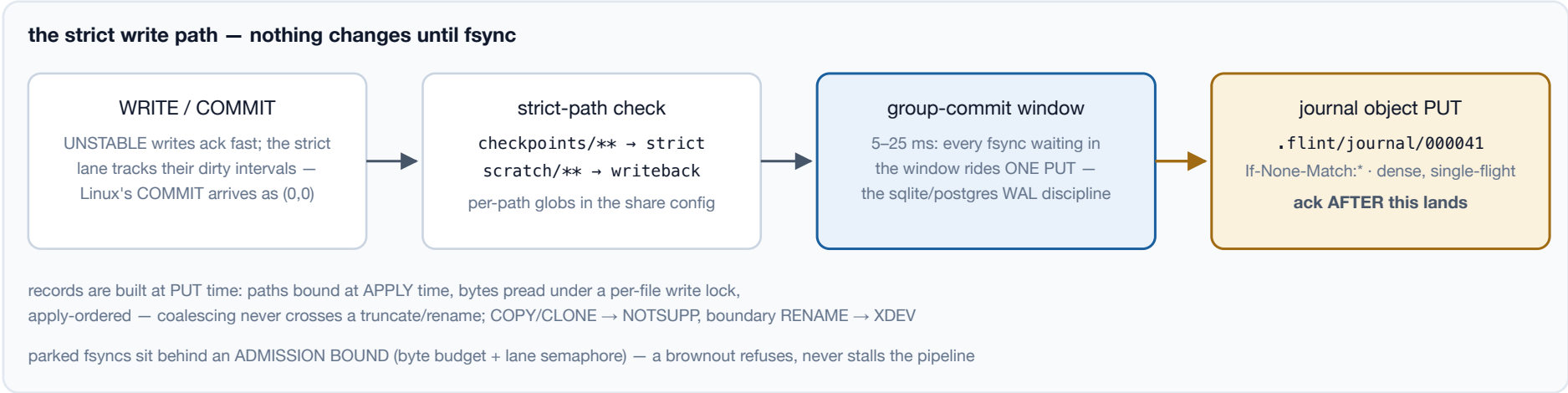


A per-share, per-path upgrade to the hub: an acked fsync is durable in regional S3 — a stronger guarantee than EFS's replication — bought with a group-committed journal beside the unchanged browsable checkpoints. The UI, the bucket format, and write-back paths are untouched; only flagged paths pay the latency.



checkpoints stay exactly what flush barriers are today

the barrier publishes whole-file generations + the manifest CAS — the BROWSABLE view, unchanged; each entry gains contained_seq — the watermark is the MIN across strict files, never a frozen global tail

(hot files legally skip flushes; a global tail would let GC delete their acked records). GC floors: lagged watermark + rebuild horizon + one full lease; strict files bypass the quiesce skip, keep the 60 s floor

restart with the PVC: replay from the fsynced applied-seq marker (usually nothing) pre-listener — page 2 removes the PVC

cost shape — the group commit is what keeps this sane

requests: one PUT per busy window — the window ADAPTS toward 25 ms under sustained load, so ~40 PUT/s ≈ \$520/mo is a design ceiling, not an accident; bursty real workloads land ~\$5–50/mo; an idle share journals nothing. SSE-KMS without Bucket Keys adds ~\$0.03/1k PUTs

(bootstrap hard-warns); a journal-scoped lifecycle rule reaps noncurrent versions and delete markers. Bytes: records chunk at 8 MiB and an op spills across batches — nothing is refused for size, COMMIT least of all. A 4 KiB commit journals ~4 KiB, never the file.

the durability ladder — where an acked fsync lives

today (write-back)
one gp3 volume, one AZ; S3 trails by flush cadence

EFS
synchronously replicated, multi-AZ

strict mode
regional S3, 11 nines, multi-AZ — ≥ EFS, per acked fsync

fsync latency: write-back ~0.1–1 ms (local) · strict on S3
Standard ~10–50 ms · strict on Express + async Standard
mirror ~2–8 ms (zone loss re-opens an AZ-scoped tail RPO)

EFS write latency is low-single-digit ms — strict is the same order for group-committed workloads

the journal fence — the honest version

create-only keys, three-arm 412 rule (higher epoch ⇒ die; equal ⇒ own lost reply; lower ⇒ replay it). Acks are LEASE-BOUNDED: pause on the first failed renew — so a paused predecessor can't ack into a GC-freed key. Termination is heartbeat-composed, not a property of the handshake alone.

The claim, precisely: with strict mode on a path, every acked fsync is durable in regional, multi-AZ S3 before the application proceeds — a strictly stronger statement than EFS makes — while unflagged paths keep today's write-back contract and the bucket's browsable view stays exactly the whole-file checkpoint view it is now. The design is three ideas, all with precedent in this codebase: **group commit** (the WAL discipline — latency is one PUT regardless of fsync rate, cost is one PUT per busy window), **read-back journaling** (a strict-lane dirty-interval tracker records UNSTABLE writes at the apply site — COMMIT arrives range-blind — and bytes are pread at PUT time under a per-file lock, paths bound at apply time), and **sequenced create-only keys** (If-None-Match:* — the same conditional-write primitive every publish already uses, composed honestly with the epoch heartbeat: acks are lease-bounded, and GC keeps lease and rebuild-horizon floors so the fence survives key deletion). What it deliberately does not change: coherence (the hub is still the single authority — strict mode is a durability knob, not a consistency change), the manifest/DR format (per-entry contained_seq, min-derived watermark), and the reader-facing contracts — an S3 browser still reads checkpoints. This page reflects **plan v2**: a 12-agent adversarial review confirmed 23 findings against v1 (docs/plans/flint-strict-plan.md is the plan of record). The one honest asterisk is Express staging: single-digit-ms fsync at the cost of an AZ-scoped tail RPO if the Express zone dies first — offered as a knob, never a default.

Once the journal exists, S3 is the authoritative log and the hub can be rebuilt anywhere from the bucket alone: checkpoint + journal replay + protocol grace. The force-detach wait and the AZ pinning hang off the hub's PVC being load-bearing — and stop existing when it isn't. (The F29 staging wedge is consumer-side CSI state; that row stays.)

the failover ladder

today — the PVC is load-bearing

pod crash ~13 s · node crash: force-detach up to ~6 min, AZ-pinned, F29 staging wedges need a manual arm

journal rebuild — successor starts ANYWHERE

no volume reattach, no AZ pinning — metadata import + strict-file pre-hydration + tail replay + grace: tens of seconds while strict files stay small (hot state) — measured by the drill, not assumed

warm standby (v2) — a second pod tails the journal continuously

takeover ≈ fence + grace; the epoch CAS + journal-seq collision already arbitrate who serves

which failure-track rows this improves — and which it does not touch

F4 power loss / F1 crash storms — acked strict writes get a second, independent durability witness

F8 disk-full, F13 quarantine fallback — recovery no longer trusts the disk that just failed

F10 node death — the ~6-min force-detach row is deleted. F29 is consumer-side CSI staging: that row STAYS

split-brain — closed for strict paths by lease-bounded acks + the seq fence (heartbeat-composed, not replaced)

NOT touched: the NFS protocol surface (conformance, quotas, deadlines — the B1–B10 work stands on its own and is a PREREQUISITE here, not a beneficiary); coherence (the hub is still the one authority); the CL load track (concurrency behavior is orthogonal to where durability lives)

strict mode is the milestone AFTER the hardening drills, standing on them — not a replacement for them

the rebuild, step by step — no PVC anywhere

- 1 schedule the pod on any node, any AZ — working set on an empty volume (now a cache)
- 2 import the last checkpoint: manifest → dirs, symlinks, evicted stubs (the existing DR verb, unchanged)
- 3 PRE-HYDRATE every strict file with tail records (the stub contract would truncate replayed bytes), then replay — contiguity-checked, guarded per-file against a checkpoint that ran ahead
- 4 fence: FENCE batch at tail+1, then LIST past it until empty — a pipelining straggler is replayed, then loses
- 5 grace runs FORCED state-lost (a fresh state.db looks innocently clean — grace would never arm): rival locks are refused, clients re-establish; B3's verifier makes un-COMMITted writes resend
- 6 serve — clients saw a stall and a reboot, never an error; acked strict writes all present

recovery ground truth is the BUCKET, not local disk — the state.db corruption and disk-full legs of the failure track get an independent witness they have never had

deliberately NOT in the journal

sessions, client records, open/lock state — rebuilt by the PROTOCOL, not by replay; in cache mode the grace is FORCED state-lost, since the reclaim tables died with the volume (NFSv4.1 specifies the rest)

write-back paths — their contract is unchanged; the journal covers strict paths only, so the tail stays small and replay stays fast

one engine, three front ends — not lite-hub-specific

the journal is tier:: functionality: the same lane is the strict mode for lean and FUSE sidecars (their fsync ⇒ S3 story on page 2 of the consistency comparison), and the lean gateway is its natural group-commit point. Build once, every front end inherits the durability knob.

out of scope: the full-pNFS/DS posture (the tier is scoped lite-only by design review A1)

what it takes to build

model first (~1.5–2 wk): the CAS heartbeat × journal handshake × GC floors × replay guards, COMPOSED — v1 scoped the handshake alone and would have verified a vacuous property. Phase A (~6–8 wk): journal writer + fence ~2k · dirty-interval tracker ~400 · hooks (ioops/perfops/fileops) ~700 · admission ~150 · policy ~200 · watermark + GC + lifecycle ~600 · drills. Phase B (~5–6 wk): cache-mode bootstrap ~1.2k · SpaceConfig byte budget ~200 · fence rework · operator ~400. Standby v2: ~2 wk.

The build, honestly costed (v2, post-review): a formal-model fortnight — the model must compose the heartbeat with the handshake, GC, and replay guards — then Phase A (~6–8 engineer-weeks) delivers the EFS-exceeding durability guarantee as an opt-in per-share knob with the PVC still present, and Phase B (~5–6 weeks) makes the PVC a cache and collapses the availability gap; a warm standby is a further ~2 weeks when wanted. The estimates grew from v1 because the adversarial review (23 confirmed findings, 8 critical — folded into docs/plans/flint-strict-plan.md) surfaced real components v1 missed: the strict-lane dirty-interval tracker, admission bounds, the per-file watermark, strict-file pre-hydration, forced-lost grace, and the cache-mode space budget. The hardening interplay, stated precisely: **B3's incarnation verifier is a prerequisite for client-transparent rebuilds; B1/B2's reclaim correctness protects PVC-mode restarts; cache mode runs grace forced-lost** — so this design stands on the hardening story rather than competing with it. The division of labor is unchanged: the CL/F drills earn the right to ship the hub at all; strict mode is the first post-GA milestone. And because the journal is tier-engine functionality, the same lane is the strict mode for the lean and FUSE sidecars later — one build, three front ends, and for a lean-first fleet the gateway is the natural group-commit point.